

作业题 16

题目

进行单中心 DLA 模型的模拟(可以用圆形边界, 也可以用正方形边界), 并用两种方法计算模拟得到的 DLA 图形的分形维数, 求分形维数时需要作出双对数图。

代码

采用盒计数法和 sandbox 法
并绘制双对数图

```
1. import numpy as np
2. import matplotlib.pyplot as plt
3.
4. aaaaa = 16807 # 递推公式参数
5. m = 2147483647 # 模数
6.
7. def azmodm(z):
8.     global m, aaaaa
9.     q = m // aaaaa
10.    r = m % aaaaa
11.    tmp = aaaaa * (z % q) - r * (z // q)
12.    return tmp + m if tmp < 0 else tmp # 返回取模后的结果
13.
14. bbbsssbbb = 0 # 初始种子值
15. aaasssaaa = 0 # 计数器
16. cccsscccc = 0 # 状态机
17.
18. def random01():
19.     global bbbsssbbb, aaasssaaa, m, cccsscccc
20.     z1, z2, z3, z4 = 668461, 816815, 116560, 4813144 # 预设的种子值
21.     if aaasssaaa == 0 and cccsscccc == 0:
```

```

22.         bbbssbbb = z1
23.     if aaasssaaa == m - 10000000 and cccssssccc == 0:
24.         bbbssbbb, aaasssaaa, cccssssccc = z2, 1, 1
25.     if aaasssaaa == m - 10000000 and cccssssccc == 1:
26.         bbbssbbb, aaasssaaa, cccssssccc = z3, 1, 2
27.     if aaasssaaa == m - 10000000 and cccssssccc == 2:
28.         bbbssbbb, aaasssaaa, cccssssccc = z4, 1, 3
29.     bbbssbbb = azmodm(bbbssbbb) # 使用递推公式生成随机数
30.     aaasssaaa += 1
31.     return bbbssbbb / m # 返回范围在 [0, 1) 的随机数
32.
33. L = 10000 # 二维数组大小
34. xy = np.zeros((L, L, 2), dtype='float64') # 存储坐标和状态的数组
35. xy[int(L/2), int(L/2)] = (1, 0) # 初始点的状态为 1, 表示已经访问
36. maxL = 1 # 记录最大半径
37.
38. def addone():
39.     global xy, maxL
40.     xy[:, :, 1] = 0 # 重置状态数组
41.     iftouch = 0
42.     fai = random01() # 生成随机角度
43.     I = np.round((maxL+5)*np.cos(2*np.pi*fai)+L/2).astype(int) # 计算新点的横坐标
44.     J = np.round((maxL+5)*np.sin(2*np.pi*fai)+L/2).astype(int) # 计算新点的纵坐标
45.     xy[I, J, 1] = 1 # 将新点的状态设置为 1
46.     while not(iftouch):
47.         a = random01()
48.         if a <= 0.25:
49.             I += 1
50.         elif 0.25 < a <= 0.5:
51.             I -= 1
52.         elif 0.5 < a <= 0.75:
53.             J += 1
54.         elif 0.75 < a <= 1:
55.             J -= 1
56.         condition = ((I-int(L/2))**2 + (J-
            int(L/2))**2 >= (maxL*5)**2) or xy[I, J, 0] == 1 or \
57.             (I <= 1) or (I == L-2) or (J < 1) or (J == L-2)
58.         if condition:
59.             fai = random01()
60.             I = np.round((maxL+5)*np.cos(2*np.pi*fai)+L/2).astype(int)
61.             J = np.round((maxL+5)*np.sin(2*np.pi*fai)+L/2).astype(int)
62.             continue
63.         iftouch = np.sum(xy[I-1:I+2, J-1:J+2, 0])
64.         if iftouch:

```

```

65.         xy[I, J, 0] = 1
66.         tmp = (I-int(L/2))**2 + (J-int(L/2))**2
67.         if tmp >= maxL**2:
68.             maxL = int(np.sqrt(tmp)) # 更新最大半径
69.
70. def change(arr):
71.     arr = np.array(arr)
72.     return arr[:, 0], arr[:, 1] # 将坐标数组拆分成横坐标和纵坐标
73.
74. def findarr():
75.     indices = np.argwhere(xy[:, :, 0] == 1) # 找到状态为 1 的点的索引
76.     return np.column_stack((indices[:, 0], indices[:, 1])) # 将横坐标和纵坐标合并成坐标数组
77.
78. n = 0
79. for k in range(60):
80.     addone()
81.     n = k + 2
82.
83. arr = findarr()
84. x, y = change(arr)
85.
86. f1 = plt.figure()
87. plt.scatter(x, y, s=1) # 绘制散点图
88.
89. sandx = []
90. sandy = []
91. for r in range(1, maxL):
92.     k = np.sum((x-int(L/2))**2 + (y-int(L/2))**2 <= r**2)
93.     sandx.append(np.log(k))
94.     sandy.append(np.log(r))
95.
96. f2 = plt.figure()
97. plt.title("sandbox")
98. plt.plot(sandx, sandy) # 绘制沙盒图
99.
100. hex = []
101. hey = []
102. for r in range(1, maxL):
103.     max_x, max_y = np.max(x), np.max(y)
104.     min_x, min_y = np.min(x), np.min(y)
105.     k = 0
106.     for i in range(min_x, max_x+r, r):
107.         for j in range(min_y, max_y+r, r):

```

```

108.         t = np.sum(xy[i:i+r, j:j+r, 0])
109.         if t >= 1:
110.             k += 1
111.         hex.append(np.log(k))
112.         hey.append(-np.log(r))
113.
114.     f3 = plt.figure()
115.     plt.title("box")
116.     plt.plot(hex, hey) # 绘制盒图
117.
118.     plt.show() # 显示图形

```

结果

3000 个点

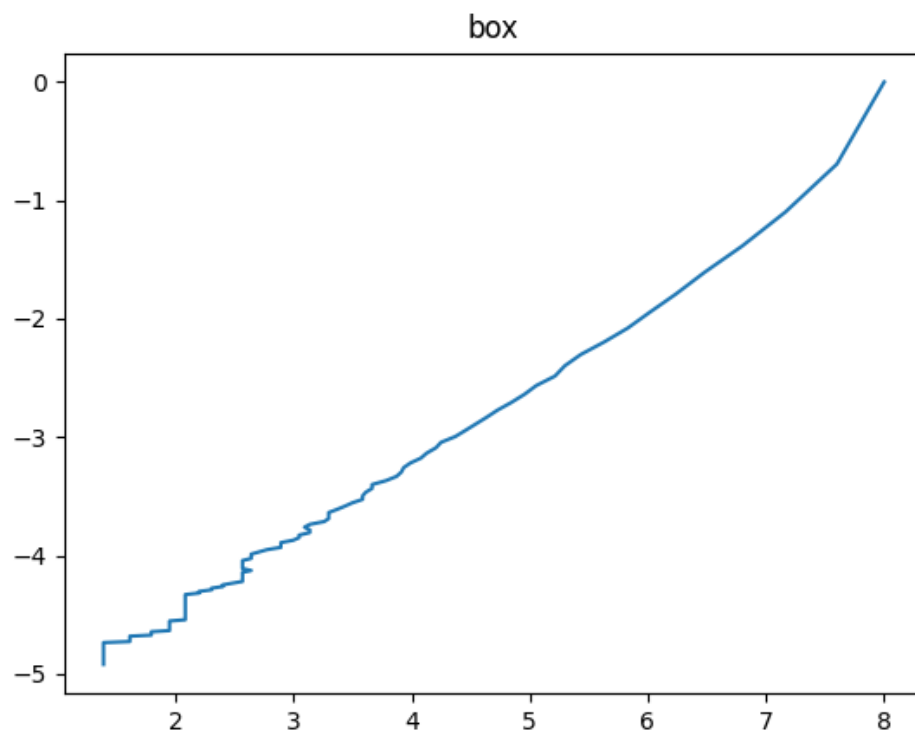


图 1 盒计数法

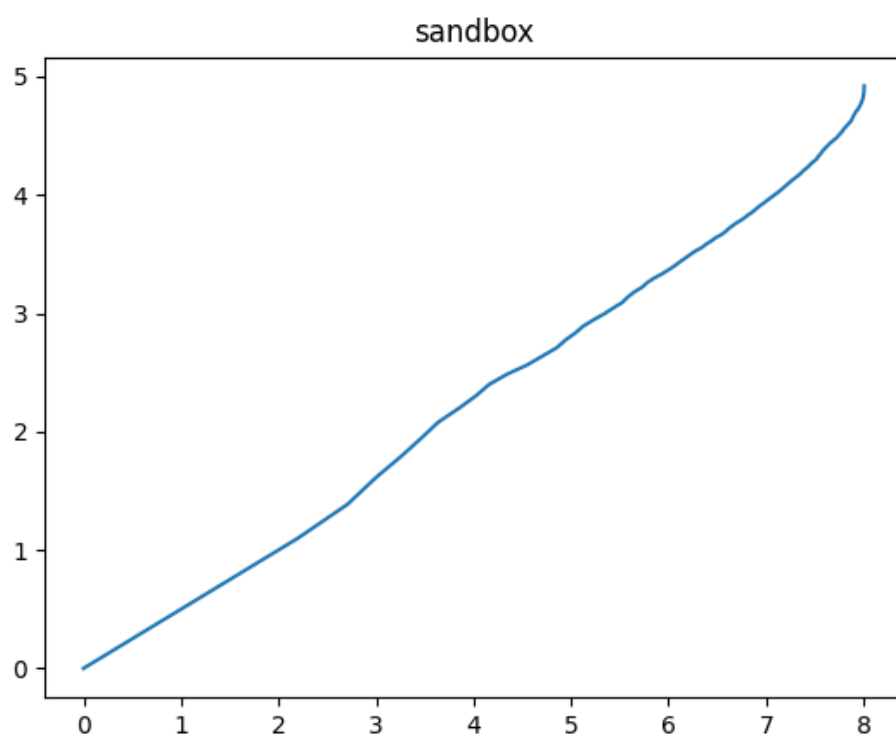


图 2 sandbox 法

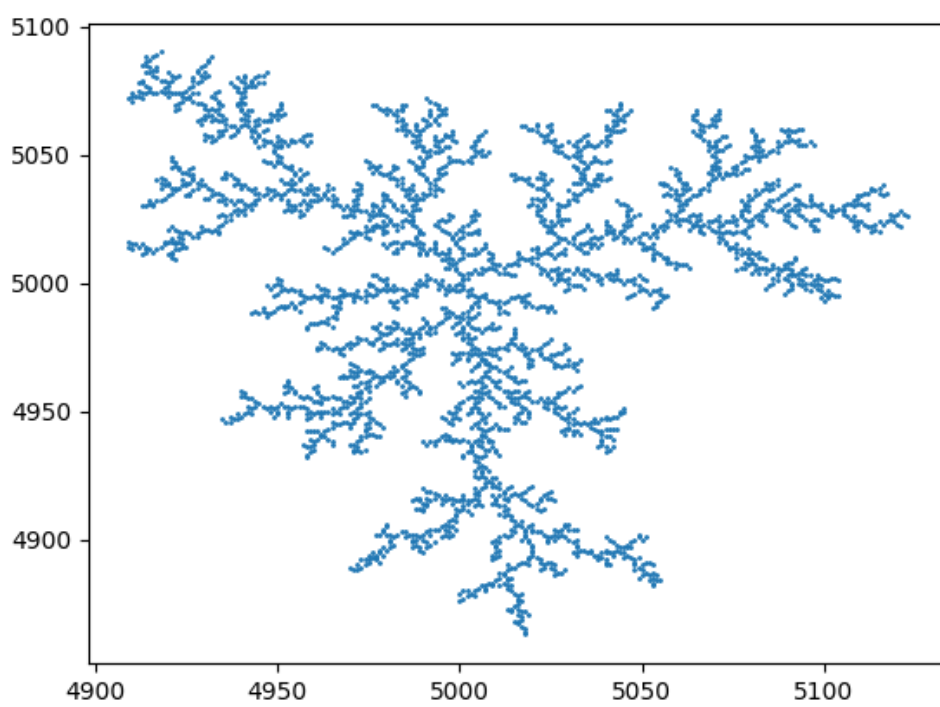


图 3 散点图

6000 个点:

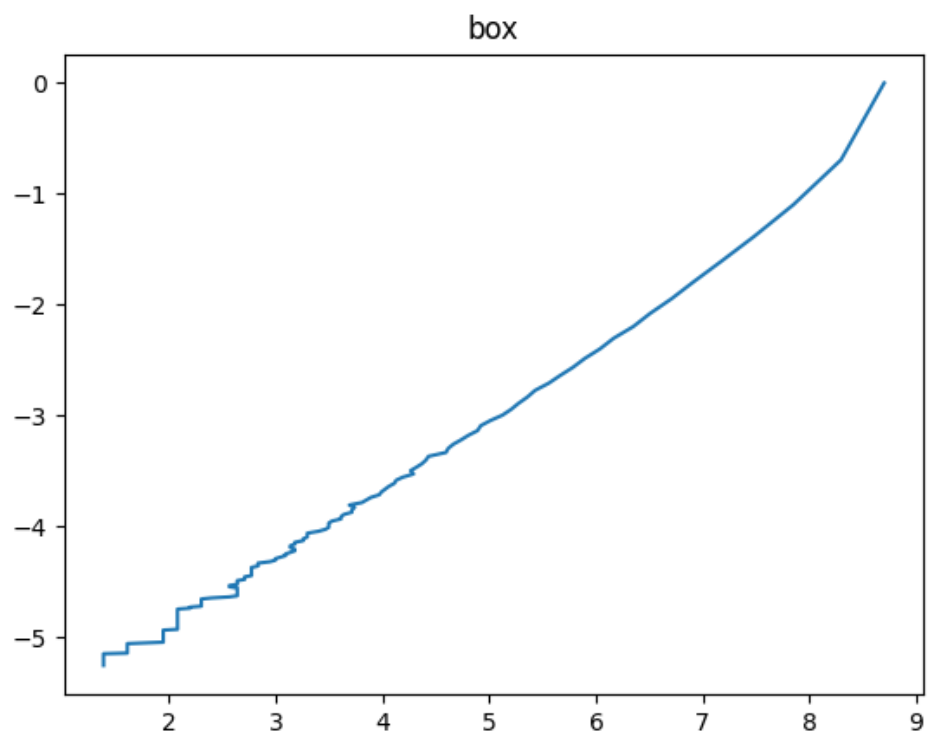


图 4 盒计数法

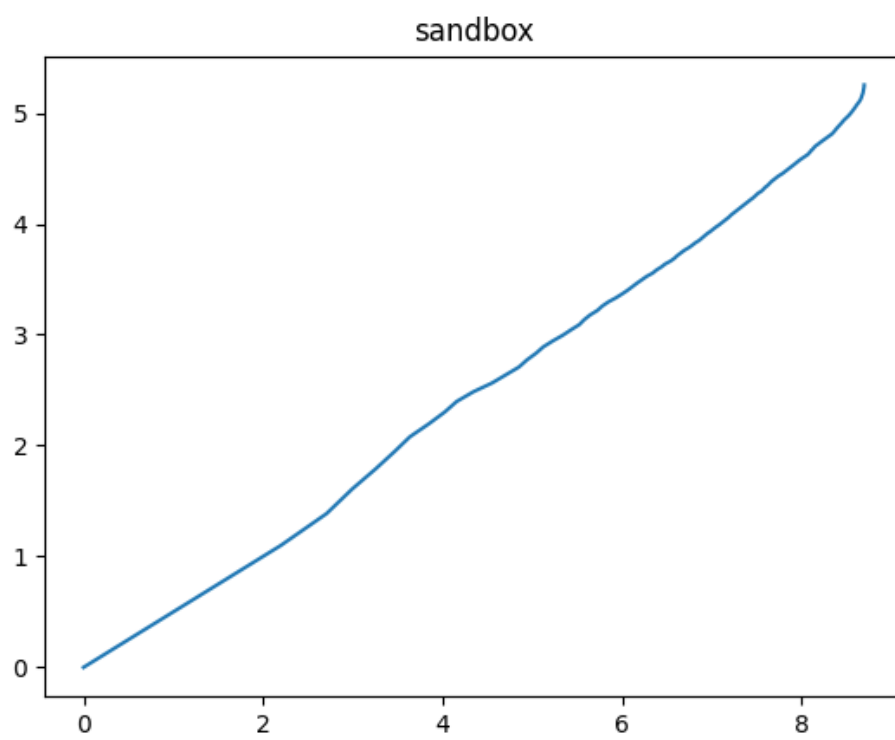


图 5 sandbox 法

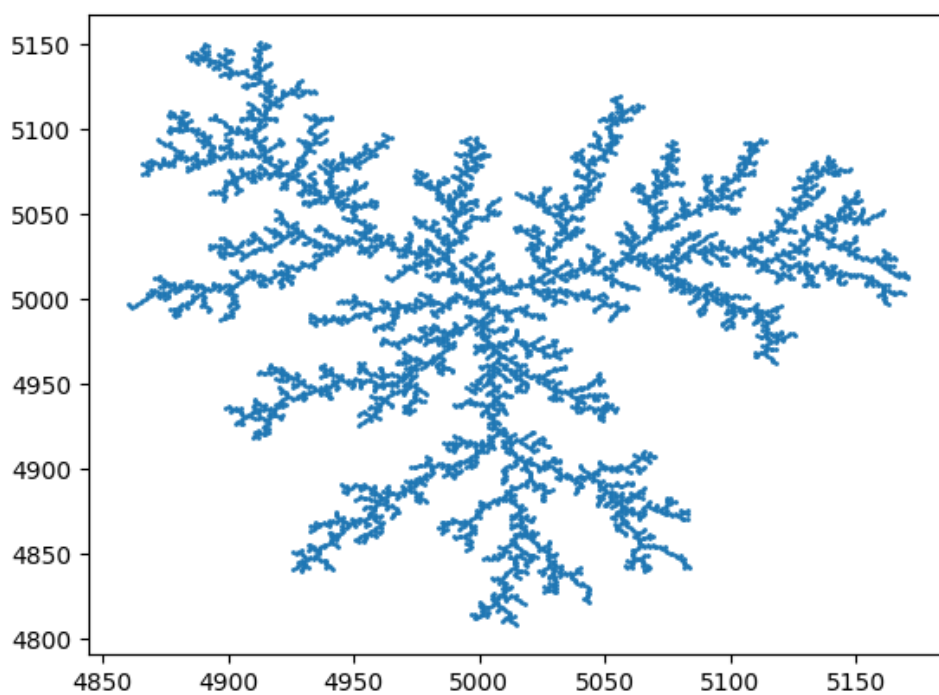


图 6 散点图

由结果可以看出，两种方法得出的双对数曲线斜率差不多，分维数为斜率的倒数 $d=8/5=1.6$